📌 Planejamento de Testes

🎯 Objetivo

Validar o fluxo de cadastro de usuários no site Automation Exercise, garantindo cobertura funcional, validações de campos e identificação de falhas críticas.

✅ Casos de Teste Executados

- ☑️ Cadastro Realizado com Sucesso → Usuário válido, fluxo completo, valida criação da conta
- ☑️ Password-Vazio → Campo senha não preenchido, valida obrigatoriedade
- ☑️ Email-Existente → Tentativa de cadastro com email já utilizado
- ☑️ Aceita-Numeros → Nome e campos textuais contendo números (validação de input)
- ☑️ Tudo-Maisculo → Dados preenchidos apenas com letras maiúsculas
- ☑️ Zipcode/Mobile-AlfanNumericos → Campos numéricos recebendo letras
- ☑️ Campo-Obrigatorio → Campos obrigatórios vazios
- ☑️ Caracter-Especial → Inserção de caracteres especiais em diversos campos

---

📊 Resultado da Execução

Resumo:

- Total de testes: 8
- Sucesso: 4
- Falha: 4

✅ Testes que passaram

- Cadastro Realizado com Sucesso
- Password-Vazio
- Email-Existente
- Campo-Obrigatorio

❌ Testes que falharam (bugs encontrados)**

- Aceita-Numeros → Sistema permitiu números em campos inválidos
- Tudo-Maisculo → Sistema não possui restrição/normalização de case
- Zipcode/Mobile-AlfanNumericos → Falha crítica de validação numérica
- Caracter-Especial → Sistema aceita caracteres inválidos

🧠 Estratégia de Teste

Tipos de Teste

- Teste funcional E2E
- Teste de validação de campos
- Teste negativo (edge cases)
- Teste exploratório controlado via massa dinâmica

Abordagem

- Data-driven (cenários centralizados)
- Reutilização via factory
- Commands customizados
- Separação por camadas (Page Object)

🏗️ Arquitetura do Projeto

```
cypress/
 ├── data/
 │   └── dados_cadastro.js
 ├── e2e/
 │   └── Cadastro/
 │       ├── cadastro.cy.js
 │       └── SignupPage.cy.js
 ├── pages/
 │   ├── CadastroPage.js
 │   └── SignupPage.js
 ├── support/
 │   ├── commands.js
 │   ├── utils.js
 │   ├── e2e.js
 │   └── factories/
 │       └── userFactory.js
```

📌 Decisões Arquiteturais

- **Page Object Pattern** → desacopla UI da lógica
- **Factory Pattern** → evita duplicação de dados
- **Custom Commands** → melhora legibilidade
- **Fixtures dinâmicas (env.json)** → controle de dados sensíveis

---

Fluxo de Execução

1. Acessa a aplicação
2. Navega até tela de login
3. Preenche signup (nome + email)
4. Gera email dinâmico (quando necessário)
5. Preenche formulário completo (se aplicável)
6. Submete cadastro
7. Valida resultado esperado

🔄 Fluxograma

```
[Início]
   ↓
[Acessa site]
   ↓
[Clica em Signup/Login]
   ↓
[Preenche Nome + Email]
   ↓
[Fluxo completo?] → NÃO → [Valida erro]
        ↓ SIM
[Preenche formulário completo]
   ↓
[Submete cadastro]
   ↓
[Valida resultado]
   ↓
[Fim]
```

📦 README - Como Executar

🔧 Pré-requisitos

- Node.js instalado
- Cypress instalado

📥 Instalação

```
npm install
```

▶️ Executar testes

```
npx cypress open
```

ou

```
npx cypress run
```

🧪 Como funciona

1. Cenários

Os testes são definidos em `dados_cadastro.js`.

Cada cenário contém:

- Massa de dados
- Tipo de fluxo
- Resultado esperado

---

2. Factory

`userFactory.js`

Responsável por gerar dados padrão e permitir sobrescrita.

---

3. Commands

`commands.js`

Centraliza ações reutilizáveis:

- cadastro de usuário
- login
- validações

---

4. Pages

Encapsulam interações com UI:

- SignupPage → tela inicial
- CadastroPage → formulário completo

---

5. Utils

- Geração de email
- Validações centralizadas

🚨 Pontos Críticos Encontrados (Visão QA Sênior)

- Sistema aceita dados inválidos (números em nome)
- Falta validação para caracteres especiais
- ZIPCODE e MOBILE sem validação numérica
- Campos obrigatórios dependem apenas do browser

👉 Tradução: backend praticamente terceirizou validação pro HTML. Clássico.

🧠 Melhorias Recomendadas

- Validar dados no backend (obrigatório)
- Criar contrato de API
- Padronizar mensagens de erro
- Implementar testes de API integrados

📄 Conclusão

A automação está bem estruturada, com boas práticas de engenharia de testes.

Mas o sistema testado... precisa de terapia.

Se isso fosse produção, já teria bug em cadastro virando incidente.